

Blatt 4

Grundlagen der IT-Sicherheit

Luis Staudt

Aufgabe 1: Signieren, verschlüsseln und entschlüsseln

a) Schlüsselpaar erstellen

Mit dem Befehl `gpg -gen-key` wurde ein Schlüsselpaar für die Adresse `lst48606@stud.hs-furtwangen.de` erstellt. Die Ausgabe von `gpg -k` zeigt beide vorhandenen öffentlichen Schlüssel:

```
luis@Alpha-17:~/Programming/GitHub/University/Basics of IT Security$ gpg -k
/home/luis/.gnupg/pubring.kbx
-----
pub   rsa4096 2025-05-17 [SC]
      FB6581778A96E5BDA32F783347D8B2C6E111735F
uid           [ unknown] Luis Staudt (GitHub GPG-Key) <luis@staudt-fridingen.de>
sub   rsa4096 2025-05-17 [E]

pub   ed25519 2026-04-23 [SC] [expires: 2029-04-22]
      A6562510D1BB06A0E3DA5F93CB4848D2C1D81365
uid           [ultimate] Luis Staudt <lst48606@stud.hs-furtwangen.de>
sub   cv25519 2026-04-23 [E] [expires: 2029-04-22]
```

Abbildung 1: Ausgabe von `gpg -k` nach der Schlüsselerstellung

Der neu erstellte Schlüssel hat den Fingerprint:

A656 2510 D1BB 06A0 E3DA 5F93 CB48 48D2 C1D8 1365

b) Öffentlichen Schlüssel exportieren

Der öffentliche Schlüssel wurde mit folgendem Befehl im ASCII-Format exportiert:

```
1 gpg --output Luis.Staudt.gpg --armor --export lst48606@stud.hs-furtwangen.de
```

Der Schalter `-armor` sorgt dafür, dass der Schlüssel als druckbarer ASCII-Text ausgegeben wird, was die Übertragung per E-Mail oder andere Textmedien vereinfacht. Die Datei `Luis.Staudt.gpg` wurde anschließend im Felix-Kurs hochgeladen.

c) Datei erstellen und signieren

Es wurde eine Textdatei `LuisStaudt.txt` mit dem Inhalt „*Ich überweise 50 Euro*“ erstellt und anschließend signiert:

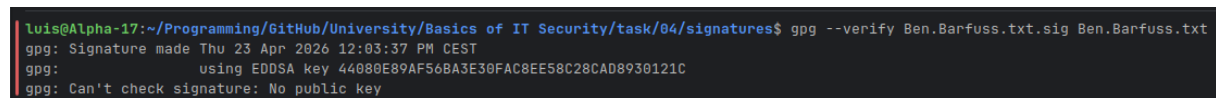
```
1 gpg --detach-sign -u lst48606@stud.hs-furtwangen.de LuisStaudt.txt
```

Dabei entsteht die Datei `LuisStaudt.txt.sig`, die die digitale Signatur enthält. Beide Dateien wurden im Felix-Kurs hochgeladen.

d) Signatur einer anderen Datei prüfen (vor Key-Import)

Mit folgendem Befehl wurde die Signatur der Datei von Ben Barfuss geprüft:

```
1 gpg --verify Ben.Barfuss.txt.sig Ben.Barfuss.txt
```



```
Luis@Alpha-17:~/Programming/GitHub/University/Basics of IT Security/task/04/signatures$ gpg --verify Ben.Barfuss.txt.sig Ben.Barfuss.txt
gpg: Signature made Thu 23 Apr 2026 12:03:37 PM CEST
gpg:                using ED0SA key 44080E89AF56BA3E30FAC8EE58C28CAD8930121C
gpg: Can't check signature: No public key
```

Abbildung 2: Signaturprüfung vor dem Import des öffentlichen Schlüssels

GPG meldet *Can't check signature: No public key*, da der öffentliche Schlüssel von Ben Barfuss noch nicht im lokalen Schlüsselbund vorhanden ist.

Die Prüfung hat zu diesem Zeitpunkt noch keine Bedeutung, da ohne den öffentlichen Schlüssel des Absenders gar keine Überprüfung möglich ist. Selbst wenn der Schlüssel vorliegen würde, wäre nicht sichergestellt, dass er tatsächlich der angegebenen Person gehört, da noch kein Vertrauen (Trust) etabliert wurde.

e) Öffentliche Schlüssel importieren

Der öffentliche Schlüssel wurde mit folgendem Befehl importiert:

```
1 gpg --import Ben.Barfuss.gpg
```

Nach dem Import wurde die Signatur erneut geprüft:

```
luis@Alpha-17:~/Programming/GitHub/University/Basics of IT Security/task/04/signatures$ gpg --verify Ben.Barfuss.txt.sig Ben.Barfuss.txt
gpg: Signature made Thu 23 Apr 2026 12:03:37 PM CEST
gpg:       using EDDSA key 44080E89AF56BA3E30FAC8EE58C28CAD08930121C
gpg: Good signature from "Ben Barfuss <bba56820@stud.hs-furtwangen.de>" [unknown]
gpg: Signature notation: manu=2,2.5+1.12,2,1
gpg: WARNING: This key is not certified with a trusted signature!
gpg:       There is no indication that the signature belongs to the owner.
Primary key fingerprint: 4408 0E89 AF56 BA3E 30FA  C8EE 58C2 8CAD 8930 121C
```

Abbildung 3: Signaturprüfung nach dem Import des öffentlichen Schlüssels

GPG meldet nun *Good signature from „Ben Barfuss“*, fügt jedoch hinzu: *This key is not certified with a trusted signature!* Die Prüfung funktioniert technisch, aber der Schlüssel gilt als [unknown].

Es kann nicht sichergestellt werden, dass der importierte Schlüssel wirklich zu Ben Barfuss gehört, da er lediglich aus dem Felix-Kurs heruntergeladen wurde, ohne die Identität des Besitzers persönlich verifiziert zu haben. Ein Angreifer könnte einen gefälschten Schlüssel unter dem Namen eines Kommilitonen hochgeladen haben.

f) Erhaltenen Schlüssel signieren

Der Fingerprint des importierten Schlüssels wurde persönlich mit dem Kommilitonen abgeglichen. Anschließend wurde der Schlüssel mit folgendem Befehl signiert:

```
1 gpg --edit-key bba56820@stud.hs-furtwangen.de
2 gpg> trust      # trust level: full
3 gpg> tsign      # trust depth: 2, no domain restriction
4 gpg> save
```

```

luis@Alpha-17:~/Programming/GitHub/University/Basics of IT Security/task/04/signatures$ gpg -k
gpg: checking the trustdb
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: depth: 0 valid: 1 signed: 1 trust: 0-, 0q, 0n, 0m, 0f, 1u
gpg: depth: 1 valid: 1 signed: 0 trust: 0-, 0q, 0n, 0m, 1f, 0u
gpg: next trustdb check due at 2029-04-22
/home/luis/.gnupg/pubring.kbx
-----
pub   rsa4096 2025-05-17 [SC]
      FB6581778A96E5BDA32F783347D8B2C6E111735F
uid           [ unknown] Luis Staudt (GitHub GPG-Key) <luis@staudt-fridingen.de>
sub   rsa4096 2025-05-17 [E]

pub   ed25519 2026-04-23 [SC] [expires: 2029-04-22]
      A6562510D1BB06A0E3DA5F93CB4848D2C1D81365
uid           [ultimate] Luis Staudt <lst48606@stud.hs-furtwangen.de>
sub   cv25519 2026-04-23 [E] [expires: 2029-04-22]

pub   ed25519 2026-04-23 [SC] [expires: 2029-04-22]
      44080E89AF56BA3E30FAC8EE58C28CAD8930121C
uid           [ full ] Ben Barfuss <bba56820@stud.hs-furtwangen.de>
sub   cv25519 2026-04-23 [E] [expires: 2029-04-22]

```

Abbildung 4: Ausgabe von `gpg -k` nach dem Signieren des Schlüssels

Ben Barfuss' Schlüssel wird nun mit `[ full ]` angezeigt. Die Signaturprüfung seiner Dateien ist jetzt vollständig gültig, da GPG dem Schlüssel vertraut.

Das Konzept des Vertrauens hängt direkt mit dem Signieren eines Schlüssels zusammen: Indem man einen öffentlichen Schlüssel signiert, bestätigt man, dass man die Identität des Besitzers persönlich geprüft hat (z. B. durch Fingerprint-Vergleich). Mit `tsign` und einer Trust-Tiefe von 2 wird zusätzlich mitgeteilt, dass man auch den Schlüsseln vertraut, die dieser Person wiederum vertraut. So entsteht ein Vertrauensnetz (*Web of Trust*).

g) Datei verändern und Signatur erneut prüfen

Die Datei `Ben.Barfuss.txt` wurde nachträglich verändert und die Signatur erneut geprüft:

```

1 echo "Geändert" >> Ben.Barfuss.txt
2 gpg --verify Ben.Barfuss.txt.sig Ben.Barfuss.txt

```

```

luis@Alpha-17:~/Programming/GitHub/University/Basics of IT Security/task/04/signatures$ gpg --verify Ben.Barfuss.txt.sig Ben.Barfuss.txt
gpg: Signature made Thu 23 Apr 2026 12:03:37 PM CEST
gpg:
gpg: using EDDSA key 44080E89AF56BA3E30FAC8EE58C28CAD8930121C
gpg: BAD signature from "Ben Barfuss <bba56820@stud.hs-furtwangen.de>" [full]

```

Abbildung 5: Signaturprüfung nach Manipulation der Datei

GPG meldet *BAD signature*. Das bedeutet, dass der Inhalt der Datei nach der Signierung verändert wurde. Die Signatur stimmt nicht mehr mit dem aktuellen Dateiinhalte überein. Dies zeigt, dass digitale Signaturen Manipulation zuverlässig erkennen.

h) Datei signieren und verschlüsseln

Die Datei wurde mit dem eigenen Schlüssel signiert und für den Empfänger verschlüsselt:

```
1 gpg --encrypt --sign --armor \
2   -u lst48606@stud.hs-furtwangen.de \
3   -r lst48606@stud.hs-furtwangen.de \
4   LuisStaudt.txt
```

Als Empfänger wurde die eigene Adresse gewählt, um den Mechanismus zu demonstrieren. Es entsteht die Datei `LuisStaudt.txt.asc`, die verschlüsselt und signiert ist.

i) Empfangene Datei entschlüsseln

Die verschlüsselte Datei wurde mit folgendem Befehl entschlüsselt:

```
1 gpg LuisStaudt.txt.asc
```

```
Luis@Alpha-17:~/Programming/GitHub/University/Basics of IT Security/task/04/signatures$ gpg LuisStaudt.txt.asc
gpg: WARNING: no command supplied. Trying to guess what you mean ...
gpg: encrypted with cv25519 key, ID 17E6E33B68618702, created 2026-04-23
" Luis Staudt <lst48606@stud.hs-furtwangen.de>"
File 'LuisStaudt.txt' exists. Overwrite? (y/N) N
Enter new filename: LuisStaudtDecrypted.txt
gpg: Signature made Thu 23 Apr 2026 01:41:39 PM CEST
gpg:      using EDDSA key A6562510D1B806A0E3DA5F93CB4848D2C1D81365
gpg:      issuer "lst48606@stud.hs-furtwangen.de"
gpg: Good signature from "Luis Staudt <lst48606@stud.hs-furtwangen.de>" [ultimate]
```

Abbildung 6: Entschlüsselung und Signaturprüfung der verschlüsselten Datei

GPG meldet *Good signature from „Luis Staudt“ [ultimate]*. Die Datei wurde mit dem privaten Schlüssel des Empfängers entschlüsselt und die eingebettete Signatur erfolgreich geprüft. Das zeigt, dass die Datei unverändert vom angegebenen Absender stammt.

Aufgabe 2: Aufbau einer Chain of Trust

a) Signierten Schlüssel exportieren

Nach dem gegenseitigen Signieren der Schlüssel wurde der öffentliche Schlüssel von Ben Barfuss inklusive der eigenen Signatur exportiert:

```
1 gpg --output Ben.Barfuss.signed.gpg --armor --export bba56820@stud.hs-furtwangen.de
```

Der Unterschied zum ursprünglich erhaltenen Schlüssel besteht darin, dass die exportierte Datei nun zusätzlich die eigene Signatur enthält. Andere Personen, die dieser Signatur vertrauen, können dadurch auch dem Schlüssel von Ben Barfuss vertrauen, ohne ihn selbst verifiziert zu haben.

b) Schlüssel des Tutors

Der öffentliche Schlüssel von `richard.zahoransky@hfu.eu` wird in der Praktikumsstunde persönlich ausgetauscht und die Fingerprints gegenseitig geprüft. Anschließend werden die Schlüssel mit `tsign` mit einer Trust-Tiefe größer 1 signiert, damit das Vertrauen an andere Kommilitonen weitergegeben werden kann.

c) Signierten Zahoransky-Schlüssel importieren

Der von einem Kommilitonen signierte öffentliche Schlüssel von `richard.zahoransky` wurde aus dem Felix-Kurs heruntergeladen und importiert:

```
1 gpg --import richard.zahoransky.signed.gpg
```

d) Signatur von ChainOfTrust.txt prüfen

Die Datei `ChainOfTrust.txt` stammt von `richard.zahoransky@hfu.eu`.

GPG vertraut dieser Signatur, obwohl die Identität von `richard.zahoransky` nicht persönlich geprüft wurde, weil ein Kommilitone dessen Schlüssel signiert hat, dem wiederum vertraut wird. Durch die Trust-Tiefe größer 1 wird dieses Vertrauen weitergereicht. GPG wertet das als ausreichenden Vertrauenspfad im Web of Trust. Dies nennt sich *Chain of Trust*: Vertrauen wird entlang einer Kette von Signaturen weitergegeben, ohne dass jeder jeden persönlich kennen muss.

Aufgabe 3: Nutzen von GPG für den E-Mail-Versand

a) Thunderbird einrichten

Thunderbird wurde installiert und das Konto `lst48606@stud.hs-furtwangen.de` eingerichtet.

b) Privaten Schlüssel exportieren

```
1 gpg -o secret-key.gpg --export-secret-key
```

c) Öffentliche Schlüssel exportieren

```
1 gpg -o pub-keys.gpg --export
```

d–e) Schlüssel in Thunderbird importieren

In den Kontoeinstellungen unter *Ende-zu-Ende-Verschlüsselung* wurde der private Schlüssel über *Schlüssel hinzufügen* aus der Datei `secret-key.gpg` importiert. Die öffentlichen Schlüssel der Kommilitonen wurden über die OpenPGP-Schlüsselverwaltung unter *Datei* → *Öffentliche Schlüssel importieren* hinzugefügt.

f) Signierte und verschlüsselte E-Mails

Nach dem Import können E-Mails in Thunderbird signiert und verschlüsselt versendet werden. Nicht einmal der Betreiber des E-Mail-Servers kann die verschlüsselten Inhalte lesen, da nur der Empfänger mit seinem privaten Schlüssel entschlüsseln kann.